

Data wrangling across columns

Agenda and reminders

- HW 2 due tonight on **GitHub classroom**
 - Commit early and often, let me know if you have any technical problems
 - Make sure to submit both the `.qmd` **and** `.md` files
- Today: data wrangling across columns

Warmup activity

Work on the activity (handout) with a neighbor, then we will discuss as a class

diamonds |>

Summarize (mean-carat = mean(carat),
sd-carat = sd(carat),

...)

Warmup

```
1 diamonds |>
2   summarize(mean_carat = mean(carat),
3             sd_carat = sd(carat),
4             mean_depth = mean(depth),
5             sd_depth = sd(depth),
6             mean_price = mean(price),
7             sd_price = sd(price))
```

A tibble: 1 × 6

	mean_carat	sd_carat	mean_depth	sd_depth	mean_price	sd_price
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	0.474	61.7	1.43	3933.	3989.

Are there any downsides to this code?

Warmup

```
1 diamonds |>
2   summarize(mean_carat = mean(carat),
3             sd_carat = sd(carat),
4             mean_depth = mean(depth),
5             sd_depth = sd(depth),
6             mean_price = mean(price),
7             sd_price = sd(price))
```

- more variables to summarize means longer code, harder to read
- requires a lot of copying and pasting
- more prone to errors when typing names of functions, variables, etc.

across: Data wrangling across columns

Instead of copying the same function multiple times for different columns, we can apply functions *across* the columns of a table:

columns to summarize

```
1 diamonds |>  
2   summarize(across(c(carat, depth, price),  
3               mean))
```

```
# A tibble: 1 × 3  
  carat depth price  
  <dbl> <dbl> <dbl>  
1 0.798  61.7 3933.
```

function to apply to each column

across: Data wrangling across columns

Instead of copying the same function multiple times for different columns, we can apply functions *across* the columns of a table:

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     mean))
```

```
# A tibble: 1 × 3
  carat depth price
  <dbl> <dbl> <dbl>
1 0.798  61.7 3933.
```

What if I want to calculate both the mean *and* the standard deviation of these columns?

across with multiple functions

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list(mean, sd)))
```

A tibble: 1 × 6

list functions to apply

	carat_1	carat_2	depth_1	depth_2	price_1	price_2
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	0.474	61.7	1.43	3933.	3989.

*} all combos of
columns & functions*

What if I want to include the function name in the summary columns?

across with multiple functions

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list("mean" = mean, "sd" = sd)))
```

A tibble: 1 × 6

	carat_mean	carat_sd	depth_mean	depth_sd	price_mean	price_sd
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	0.474	61.7	1.43	3933.	3989.

Handwritten annotations:
An arrow labeled "name" points from the word "name" to the string "mean" in the list function.
An arrow labeled "function" points from the word "function" to the function "mean" in the list function.

What if I want to change the order of the column names (e.g. mean_carat vs. carat_mean)?

across with multiple functions

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list("mean" = mean, "sd" = sd),
4                     .names = "{.col}_{.fn}")) ← how do we combine?
```

A tibble: 1 × 6

	carat_mean	carat_sd	depth_mean	depth_sd	price_mean	price_sd
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	0.474	61.7	1.43	3933.	3989.

extract col name extract function name

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list("mean" = mean, "sd" = sd),
4                     .names = "{.fn}_{.col}"))
```

A tibble: 1 × 6

	mean_carat	sd_carat	mean_depth	sd_depth	mean_price	sd_price
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	0.474	61.7	1.43	3933.	3989.

Summarizing more columns

```
1 diamonds |>
2   summarize(across(c(carat, depth, price),
3                     list("mean" = mean)))
```

A tibble: 1 × 3

	carat_mean	depth_mean	price_mean
	<dbl>	<dbl>	<dbl>
1	0.798	61.7	3933.

How would I modify this code to calculate the mean for *all* the numeric variables (carat, depth, table, price, x, y, z)?

Summarizing more columns

Option 1:

```
1 diamonds |>
2   summarize(across(c(carat, depth, table, price, x, y, z),
3                     list("mean" = mean)))
```

A tibble: 1 × 7

	carat_mean	depth_mean	table_mean	price_mean	x_mean	y_mean	z_mean
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	61.7	57.5	3933.	5.73	5.73	3.54

Are there any issues with this approach?

Efficiently summarizing more columns

```
1 diamonds |>
2   summarize(across(
3     where(is.numeric),
      list("mean" = mean)))
```

find columns that satisfy a condition

checking if column is a numeric variable

A tibble: 1 × 7

	carat_mean	depth_mean	table_mean	price_mean	x_mean	y_mean	z_mean
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	61.7	57.5	3933.	5.73	5.73	3.54

Efficiently summarizing more columns

```
1 diamonds |>
2   summarize(across(where(is.numeric),
3                     list("mean" = mean)))
```

A tibble: 1 × 7

	carat_mean	depth_mean	table_mean	price_mean	x_mean	y_mean	z_mean
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.798	61.7	57.5	3933.	5.73	5.73	3.54

`where(is.numeric)` returns the columns which are numeric:

```
1 is.numeric(diamonds$carat)
```

```
[1] TRUE
```

```
1 is.numeric(diamonds$price)
```

```
[1] TRUE
```

```
1 is.numeric(diamonds$clarity)
```

```
[1] FALSE
```

Efficiently summarizing more columns

We can use `where` with other functions too. For example:

```
1 diamonds |>
2   summarize(across(where(is.factor),
3                     list("num_categories" = n_distinct)))
```

find the categorical variables

A tibble: 1 × 3

	cut_num_categories	color_num_categories	clarity_num_categories	(number of distinct values)
1	5	7	8	

↑ number of categories

What do you think this code is doing?

Class activity

- Work with a neighbor on the class activity
- At the end of class, submit your work as an HTML file on Canvas (one per group, list all your names)